The State of Agentic AI Evaluation

Key findings from the Hugging Face Agentic Evals Workshop. Five speakers, five perspectives on why evaluation must evolve. From eval transparency to reliability gaps to dynamic benchmarks — this is what the community found.

01 Eval Transparency
What's hidden in the fine print

02 The Reliability Gap
Why capability benchmarks miss the point

03 Dynamic Benchmarks
Testing agents in changing environments

04 Environment Design
Sandbox-first evaluation methodology

05 Community Standards
Open, decentralized, reproducible evals

10 panels total

Abijit Kosh	Arvind Narayanan	Pierre-André	Mahesh	Nathan
HF / Eval Coalition	Princeton University	Meta / ICLR 2026	Bespoke Labs	HF / Open LLM Leaderboard

SPEAKER 1 / ABIJIT KOSH, HUGGING FACE

The Eval Crisis: Reporting Problems

Evaluation nuances are hidden in the fine print. Model developers are becoming less transparent, not more.

40 / 237

Problems omitted from OpenAI's SWE-bench score (tiny fine print)

<15%

Of model releases mention labor or environmental costs

2022

Transparency declining since then

SPECIFIC FAILURES

OpenAI GPT-5.2

Reported high SWE score but omitted 40 of 237 problems in tiny fine print. Criticized by Anthropic and eval researchers.

Meta Llama 4

Used different model versions to score different benchmarks. Benchmark maxing: optimizing for the score rather than the underlying capability.

Chart manipulation

69.1 and 30.8 displayed as similarly tall histograms. Unscaled axes and missing error bars.

Declining teams

Companies have broken up or reassigned teams dedicated to documentation and social impact evaluation.

> *Positive signal: third-party evaluations are increasing in both quality and quantity.*

Source: Abijit Kosh, Hugging Face / Eval Coalition

What's Missing + Solutions

6 GAPS IN AGENT EVALS

Session-level data
Identical scores can mask completely different agent behavior and trajectories

Agent identity
Model is not the agent: sub-agents, MCP servers, and memory config all matter

Robustness standards
No norms for seeds, prompt perturbations, or pass@K across benchmarks

Cost reporting
Token usage, API costs, and latency are rarely disclosed alongside scores

Generalist compatibility
Eval harnesses assume specific tool interfaces that block generalist agents

Human interaction
Real workflows involve back-and-forth, but evals test agents in isolation

EVERY EVAL EVER PROJECT

A unified open data format collecting all first and third-party evaluations on Hugging Face. Schema requires source provenance, model specification, evaluation library, and instance-level results.

PROPOSED AGENTIC EXTENSIONS

01 System Composition
Models inside the system, their roles, sub-agents, MCP servers

02 Session Semantics
Fields defining what constitutes a particular run or session

03 Interaction Accounting
Measurements of every interaction in the agent workflow

04 Eval Conditions
Conditions needed to reproduce an agent action by a third party

POLICY IMPLICATIONS

> *Governance requires session data. Just giving a score is not enough to audit agent behavior. White-box system records are needed to identify which actor was responsible for a harm.*

Source: Abijit Kosh, Hugging Face / Eval Coalition

Capability ≠ Reliability

AI agents crush capability benchmarks. Yet there is no measurable GDP impact. What's going on?

REAL-WORLD RELIABILITY FAILURES

Rabbit R1

Delivered food to the wrong address. With pre-LLM systems, playing the wrong song is an annoyance. With agents using your credit card, 10% failure is dead on arrival.

OpenAI Operator

Made incorrect online purchases using real payment methods. Users reported wrong items, wrong quantities, and unexpected charges on their accounts.

Agentic coding

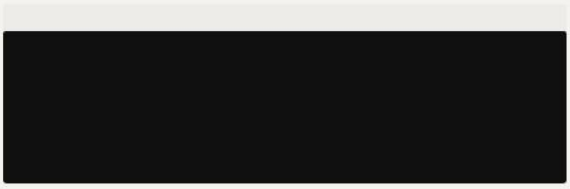
A coding agent deleted a production database during what was supposed to be a routine bug fix. The kind of error that's catastrophic, not recoverable.

Governments

Government systems adopted AI agents despite usually being slow to move. Even they aren't immune from reliability failures in deployed systems.

> *Ever since the Wright brothers, we've demonstrated planes can fly. Getting to one error per trillion miles took most of a century. AI agent reliability may face a similar timeline.*

OVER 18 MONTHS / 14 FRONTIER MODELS



~85%

Capability
Can it do the task at all?

~32%

Reliability
Does it do it consistently?

Source: Arvind Narayanan, Princeton University

Reliability: Deep Dive

4 DIMENSIONS, 12 METRICS

Consistency

1a. **Outcome** — same pass/fail each run?
1b. **Trajectory** — same action sequence?
1c. **Cost** — stable resource usage?

Robustness

2a. **Fault** — inject API timeouts, errors
2b. **Prompt** — reworded input, same answer?
2c. **Context** — noisy or missing context?

Calibration

3a. **Calibration** — confidence vs reality. Improving.
3b. **Discrimination** — separate wins from losses? Getting worse.

Failure Severity

4a. **Severity** — formatting errors vs data deletion
4b. **Reversibility** — can the harm be undone?
4c. **Scope** — one user or system-wide?

TRACE ANALYSIS INSIGHTS

- Models confuse clean process with correct answer — messy tool-calling creates false underconfidence
- Ambiguous questions are valuable — real tasks are always ambiguous
- Hallucinations spike when faults are injected — model fabricates instead of failing

> *When benchmarks "saturate," maybe it's the metric that's saturated, not the task suite. Planning a Reliability Index to track all 12 metrics over time.*

Source: Arvind Narayanan, Princeton University

GAIA 2: The Benchmark

Real agents work in a changing world. Emails arrive, prices shift, meetings get cancelled. GAIA 2 simulates interconnected apps where the world changes during the task.

WHY SIMULATION?

01 Reproducible
Observable evaluation, test robustness reliably

02 Safe
Test destructive actions (delete emails, cancel events) without risk

03 Cheap
No external API dependencies, no bandwidth costs

04 Controllable
Inject faults, vary API signatures, add noise at will

1,000	10	11
Scenarios	Universes	Apps per universe

BUILT ON MARE FRAMEWORK

APPS → UNIVERSES → EVENTS → SCENARIOS

Apps
Stateful services (email, messenger, calendar, files) with API, MCP, and CLI access

Universes
Simulated environments with auto-generated personas, past emails, calendar events, message history

Events
Injected from user, agent, or environment mid-task. New emails arrive, meetings get cancelled, prices change while the agent works.

Scenarios
Task prompt + expected event sequence + environment changes during task

GAIA 2: Results & Verification

5 CAPABILITY DIMENSIONS

Execution: multi-tool tasks. Search: find info across apps. Adaptability: react when environment disrupts. Time: respond to time-based triggers. Ambiguity: know when to ask clarifying questions.

Time-based tasks: near zero percent success across all frontier models.

VERIFICATION APPROACH

Hard Verifiers

Compare expected DAG of actions vs actual DAG. Event-by-event: correct order, parameters, recipients, timing. Pure equality or simple algorithmic checks. Cheap, reliable, reproducible.

Soft Verifiers

LLM-as-judge for generated content (e.g., does the email body match the expected intent?). Used only where hard verification isn't possible. More expensive and less deterministic.

> *Moved away from rubric-based judging — too expensive and too dependent on LLM capability. Hard verifiers are the default.*

ROBUSTNESS TESTING

TOOL FAILURE INJECTION | API SIGNATURE VARIATION | ENVIRONMENT NOISE | AGENT-TO-AGENT DELEGATION

How to Evaluate: Methodology

Like Hiccup in How to Train Your Dragon — the real transition happens when you start to observe and truly understand your agent before trying to train it.

COMMON MISTAKES

- ✖ Checking only the final output — breaks when anything changes
- ✖ Starting with granular function call details too early
- ✖ Deploying to production and evaluating there

WHY AGENT EVALS ARE HARD

Stochastic
Two identical runs can produce completely different results. Temperature, tool selection, and timing all introduce variance.

Multi-step
Many actions to reach the final outcome, each introducing complexity

World-changing
Agent actions alter the environment, making evals non-reproducible

Expensive
Real-world interactions cost money and risk destructive actions. Running N rollouts per task across multiple models adds up fast.

TWO LEVELS OF EVALUATION

01 Verifiable outputs
Did the unit tests pass? Is the math correct? Is the bug fixed? Gotcha: don't weight all tests equally — many simple tests can mask important failures. Watch for reward hacking: the agent fixes the test instead of the bug.

02 Non-verifiable outputs
For tasks like deep research where there's no single right answer. Define rubrics with weighted dimensions scored 0–1. Use LLM-as-judge. Get a composite score you can track over time.

How to Evaluate: The Stack

CRITICAL DESIGN RULES

- Environment must mirror production as closely as possible
- Agent must NEVER have access to the grader or solution (prevents reward hacking)
- Grade ALL requested outcomes — if you grade 2 of 3 things, the agent ignores the third
- Add fingerprints to unit tests to prevent agent manipulation
- Same model on different harnesses gives drastically different results (visible on Terminal Bench dashboard)

THE 6-STEP EVALUATION STACK

01 Build sandbox environment
Mirror production with Docker, Harbor, or OpenEnd

02 Define representative tasks
Reflective of real workloads, varying difficulty

03 Set up graders
Hard verifiers where possible, rubrics for open-ended

04 Measure success rate / pass@K
N rollouts, compare across models and harnesses

05 Analyze traces
Automated failure analysis for hidden patterns

06 Document everything
System composition, session semantics, eval conditions

METRICS

SUCCESS RATE | PASS@K | STEPS | TOKENS | LATENCY | COST | SAFETY

> *Once environment, tasks, and graders are solid, run RL algorithms like GRPO to optimize. Measure improvements, then deploy.*

SPEAKER 5 / NATHAN, HUGGING FACE

Community Evals + Call to Action

COMMUNITY EVALS ON THE HUB

- Anyone can contribute — decentralized evaluation at scale
- eval.yaml defines: benchmark name, evaluation framework (e.g., Inspect AI), tasks, solvers (how to prompt the model), scorer (how to grade the answer)
- One command to run: use Inspect AI with HF dataset references
- Results as PRs on model repos — community discusses, model authors can hide disputed scores
- 13+ benchmarks live: SWE-Bench, Terminal Bench, HLE, AM 2026, and more

Never evaluate via inference providers. You evaluate the provider's configuration, not the model itself.

OPEN CHALLENGES

Long horizon evaluation	UNSOLVED
Multi-agent evaluation	EARLY STAGE
Sim-to-real gap	ACTIVE RESEARCH
Benchmark gaming	NEEDS STANDARDS
Reliability optimization	NOT GUARANTEED

THE CALL TO ACTION

- → Build evals in the open
- → Stop benchmark chasing
- → Invest in scalable oversight
- → Use open reporting frameworks
- → Measure reliability, not just capability
- → Keep humans in eval policy

> *Whether a benchmark is public or private has no strong correlation with how quickly it saturates. The real challenge is multi-agent evaluation — adding more agents creates an explosion of parameters.*